BRAINHACK 2026

RoboVerse Supplementary 1

AGENDA

Main goal: To cover terms or concepts that are briefly covered in other workshops.

1. **Coordinate Frames**
1. **Mapping and point cloud for path finding**

Coordinate Frames

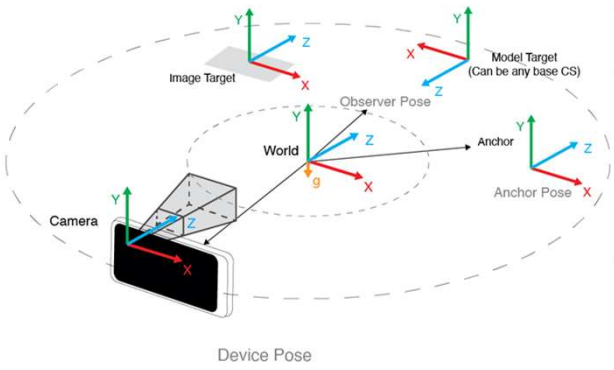
Why Coordinates Frames matter?

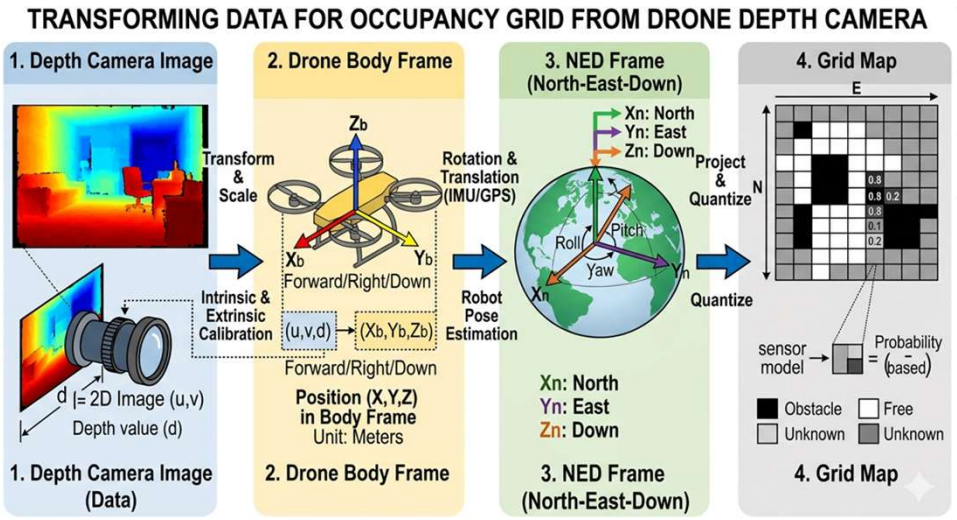
1. Different sensors (such as an IMU or a camera) output data in x, y, z coordinates. However, the meaning of these axes depends on the sensor's own coordinate frame, so the same x, y, z values can correspond to very different directions of movement.

- We need to integrate the data from these different sensors and use it to control the quadcopter.

- We need a way to transform the data across the different reference of X,Y, Z so that we could create an Occupancy Grid from depth camera. That requires to align these frames:
  - Depth Camera Image -> Drone Body Frame -> NED Frame -> Grid Map

- VIO estimates position and orientation in its own coordinate frame. To use this data correctly, you must transform it into the target coordinate system. Wrong transformations cause misalignment and infinite drift.



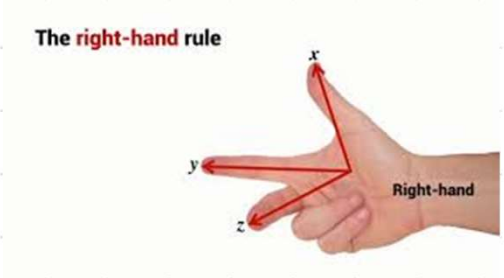


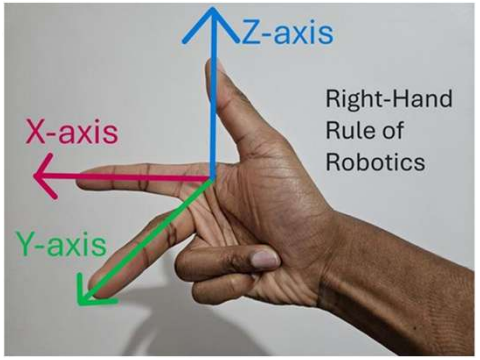
Slide 4

1 I've tweaked the phrasing to make it clearer.
Terry Sim, 27/4/2026

Coordinates Frames

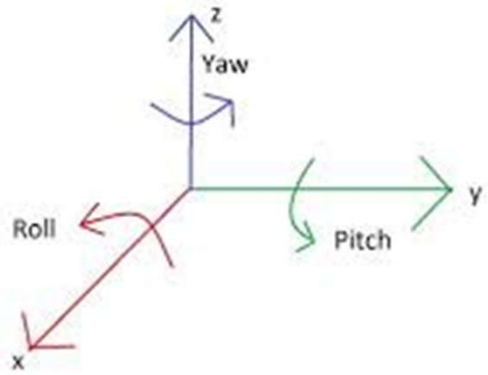
- A 3D coordinate frames consists 3 axes perpendicular to each other and intersect at a common point.
  - X-axis, Y-axis and Z-axis
- Different components might use the 3 axes to describe different direction of motions and rotations.
- Right-hand is often used to describe the orientation of the 3 axes, using thumb, index finger and middle finger.
- It is often important to find out what does X-axis, Y-axis and Z-axis for different components and context.
  - E.g. NED means X is describing forward/backward, Y is right/left, Z is down/up.





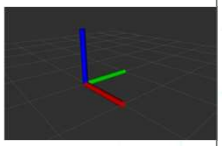
Rotation

- Rotation are often describe using Euler angles which are roll, pitch and yaw.

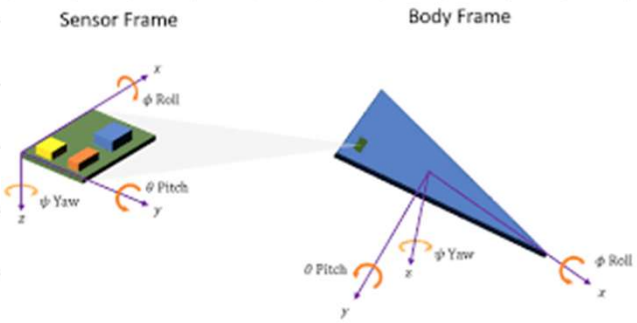


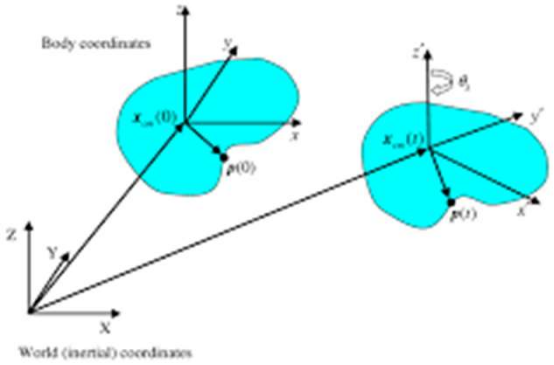
Common Coordinate Frames

- World coordinate frame:
  - global reference point for all robots, drones and objects in a given environment.



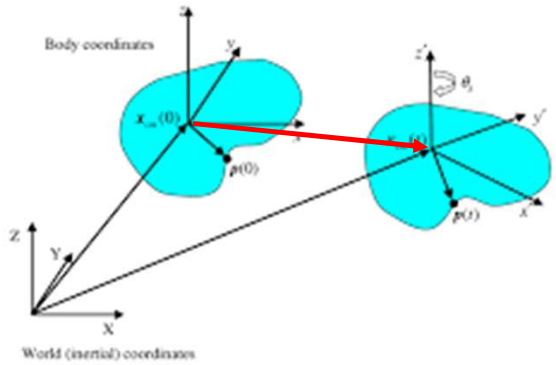
- Body frame:
  - Reference point attached to center of an object. The object can be center of drone, center of a sensor, etc.
- World coordinate frame is like the “ground truth” of the environment.
- All other coordinate frames are ultimately referenced back to world frame, allowing all drones and its sensors (camera and IMU) to understand each other’s positions and coordinate actions.





Representing coordinates

- Points are often represented as $\begin{bmatrix} x \\ y \end{bmatrix}$ or $\begin{bmatrix} x \\ y \\ z \end{bmatrix}$
- Matrices are used to represent points as column vectors and coordinate frames.
- Now, let's say we know that object A is x, y, z (red arrow) from object B. Object B is 10,10,10 in the world coordinate frame. If we wish to know what is the coordinate of Object B in world coordinate frame, we need to understand coordinate transformation.
  - See video to understand more: https://www.youtube.com/watch?v=H_94DTWd8ck
  - Or this hack video (go to time 5:44): https://youtu.be/tAu8-gkxAcE?si=NI2MaA65g14miRVL
- The Hack video is useful to grasp the technique to create transformation matrix which will come in handy when configuring VIO tools like openvins.



Transforming coordinates in python

- Coordinate transformation can be easily handled in python using numpy. This is coding example. Sometimes you need to calculate manually.
- Explanation of codes:
  - Homogeneous Coordinates: To combine rotation (multiplication) and translation (addition) into one operation, we use 3x3 matrices instead of 2x2. This requires adding a 3rd dimension to the points (a column of ones).
  - Transformation Matrix (T): The matrix defines the rotation and translation simultaneously. The upper-left 2x2 part is for rotation, and the top-right 2x1 column is for translation.
  - Matrix Multiplication (T @ points_h.T): applies the transformation matrix to all points at once. Using the @ operator is the preferred method for matrix multiplication in NumPy.
  - Efficiency: Instead of looping through each point, we represent all points as a single array (N, 2) and use NumPy broadcasting and linear algebra routines for high performance.

```python
import numpy as np

def transform_2d_points(points, angle_deg, translation_vec):
    """
    Applies rotation and translation to a set of 2D points.
    points: np.array of shape (N, 2)
    angle_deg: rotation angle in degrees
    translation_vec: [tx, ty]
    """
    # 1. Convert angle to radians
    theta = np.radians(angle_deg)
    c, s = np.cos(theta), np.sin(theta)

    # 2. Define the Transformation Matrix (3x3 Homogeneous)
    # R = [[cos, -sin, 0],
    #      [sin,  cos, 0],
    #      [0,    0,   1]]
    T = np.array([
        [c, -s, translation_vec[0]],
        [s,  c, translation_vec[1]],
        [0,  0, 1]
    ])

    # 3. Convert points to Homogeneous Coordinates (N, 3)
    # Add a column of ones to make it (N, 3)
    ones = np.ones((points.shape[0], 1))
    points_h = np.hstack([points, ones])

    # 4. Apply Transformation
    # Transpose points to (3, N) for matrix multiplication: T @ P.T
    # Then transpose back to (N, 3)
    transformed_h = (T @ points_h.T).T

    # 5. Convert back to (N, 2)
    return transformed_h[:, :2]

# --- Example Usage ---
# Define points: (0,0), (1,0), (1,1)
points = np.array([[0, 0], [1, 0], [1, 1]])

# Rotate 90 degrees and move x+2, y+1
new_coords = transform_2d_points(points, 90, [2, 1])

print("Original Points:\n", points)
print("Transformed Points:\n", new_coords)

```

1. Mapping and point cloud for path finding

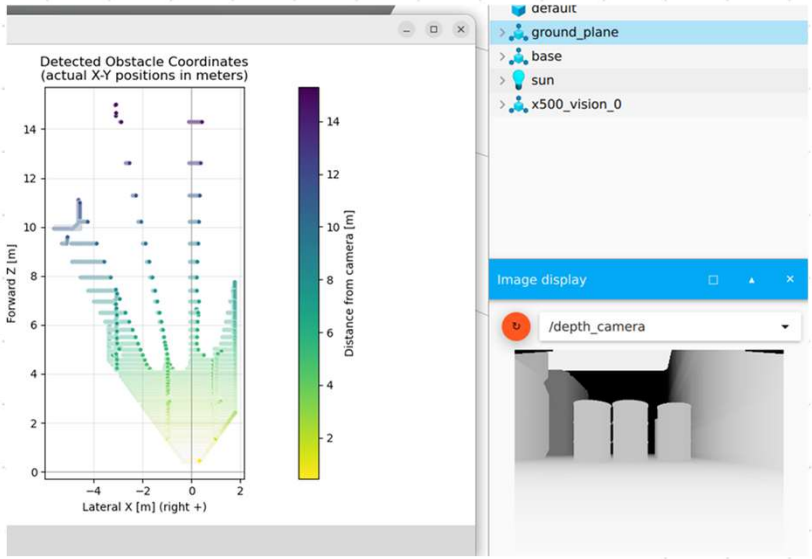
Generating global map for path finding

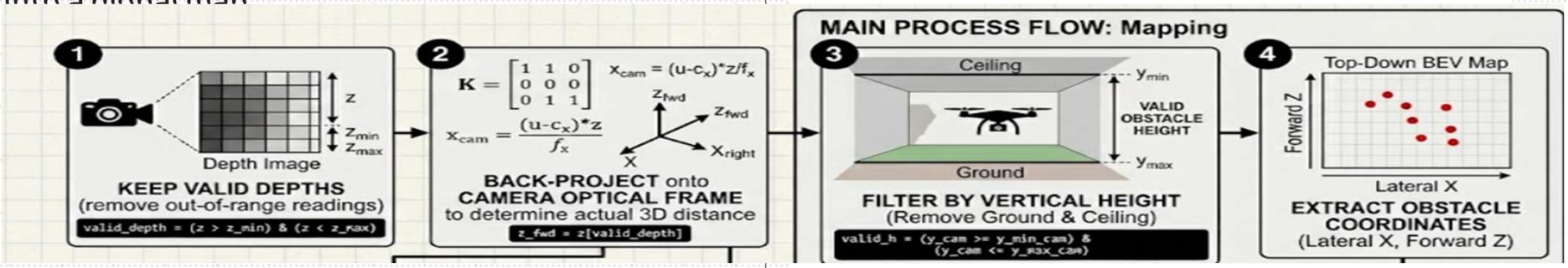
- Turn depth image at each drone position into a local obstacle map.
- Then combine many local maps into a global map.

PLEASE refer to top_down.py

Using depth image to generate a top-down local map

- Leverage depth image to remember the position of obstacles, using a top-down local map.
- A local map that describes from the perspective of drone's current position.
- The function `depth_to_xy_map` in `top_down.py` process the depth image and output top down map using the camera intrinsic to determine the distance of obstacle.
- As the drone avoid the obstacles, repeatedly call this function and combine the map output into a global map





PLEASE refer to GlobalMapper.py

Extending the depth_to_xy_map function into a global mapper

- Globalmapper.py uses depth_to_xy_map to combine local maps into a global map.
- First, GlobalMapper transform the coordinates of obstacles in a local map into global NED frame. Then add that local map to the global_map.
- Within the drone control loop, call get_frame to get depth_image and then call update_frame with that depth_image.

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap, BoundaryNorm
from top_down import depth_to_xy_map
from depth_receiver import DepthReceiver
import time
from drone_control import Drone
import asyncio
from get_position_with_task import SharedState, position_monitor_task, run


class GlobalMapper:
    """
    Incremental top-down occupancy grid mapper using NED (North-East-Down) pose.

    Coordinate Conventions:
    - Pose: {'north': float, 'east': float, 'yaw': float}
    - Yaw: radians, clockwise from North (standard NED heading)
    - Camera: Forward-facing, level mount assumed (X_cam=right, Z_cam=forward)
    - Grid: row=North, col=East (matches standard map orientation)
    """
    def __init__(self, K,
                 cam_height=1.0, obs_h_min=0.1, obs_h_max=1.5,
                 z_min=0.2, z_max=15.0,
                 yaw_in_degrees=False, yaw_clockwise=True, yaw_smoothing=0.7):
        self.K = K
        self.cam_height = cam_height
        self.obs_h_min = obs_h_min
        self.obs_h_max = obs_h_max
        self.z_min = z_min
        self.z_max = z_max

        # Yaw handling
        self.yaw_in_degrees = yaw_in_degrees
        self.yaw_clockwise = yaw_clockwise
        self.yaw_smoothing = yaw_smoothing
        self.last_yaw_rad = 0.0
        self.first_frame = True

        # Global point storage: (N, 2) array [north, east] in meters
        self.global_points = np.empty((0, 2), dtype=np.float32)

    def _local_to_ned_global(self, local_xy, north, east, yaw_rad):
        """Transform local (X_cam=right, Z_cam=forward) to NED global (north, east)"""
        X_cam = local_xy[:, 0]
        Z_cam = local_xy[:, 1]
        c, s = np.cos(yaw_rad), np.sin(yaw_rad)

        north_global = north + Z_cam * c - X_cam * s
        east_global  = east  + Z_cam * s + X_cam * c
        return np.column_stack([north_global, east_global])

    def update_frame(self, depth_img, pose):
        north = pose['north']
        east = pose['east']
        raw_yaw = pose['yaw']

        # 1. Yaw conversion & smoothing
        if self.yaw_in_degrees:
            raw_yaw = np.deg2rad(raw_yaw)
        if not self.yaw_clockwise:
```

PLEASE refer to PointCloudMapper.py

Before building a path finder

- Now, your code would have able to map the area and make the unknown known. It's time to use the known area map to guide drones to go from point A to point B using a path finder.
- The purpose of path finder is to find a series of waypoints to travel between two points without colliding into any obstacle.
- But our GlobalPlanner do not mark "free space". With free space, our drone can hop through the free space towards the target.
- We need something that uses the Global map to check if a point is near (< safety margin) or is an obstacle point. That's PointCloudMapper.py. It simply uses scipy.spatial.KDTree for O(log N) collision queries.
- See screenshot on how to use GlobalMapper and PointCloudPlanner.

```python
# 1. Initialize mapper & planner
mapper = GlobalMapper(K, yaw_in_degrees=True, yaw_smoothing=0.8)
planner = PointCloudPlanner(safety_margin=0.6)  # 60cm drone + buffer

# 2. In your control loop:
for step in range(steps):
    # ... get depth + pose ...
    mapper.update_frame(depth_img, pose)

    # Update planner's collision map every N frames
    if step % 5 == 0:
        pts = mapper.get_global_points()
        # Optional: define mission bounds so planner knows where free space ends
        bounds = (-5, 50, -10, 10)  # north_min, north_max, east_min, east_max
        planner.update_map(pts, bounds)

    # 3. Planning step (pseudo-RRT* collision check)
    sample_point = np.array([current_north + np.random.uniform(-2, 2),
                             current_east + np.random.uniform(-2, 2)])
    if planner.is_collision_free(sample_point):
        # Safe to extend trajectory toward sample_point
        pass
```

PLEASE refer to RRTStarPlanner.py

A Path Finder

- With GlobalMapper and PointCloudPlanner, you can build your own path finder using popular path finding algorithm like RRT, shortest path and etc.
- Keep using PointCloudPlanner's is_collision_free to check for every possible point generated by your path finding algorithm.
- RRTStarPlanner.py is one such Path Finder implementation.
- See right on how to integrate GlobalMapper and RRTStarPlanner.

```python
if __name__ == "__main__":
    K = np.array([[433.0, 0.0, 320.0],
                  [0.0, 433.0, 240.0],
                  [0.0, 0.0, 1.0]])

    # 1. Build map
    mapper = GlobalMapper(K, yaw_in_degrees=True, yaw_smoothing=0.8)
    for i in range(60):
        t = i * 0.25
        pose = {'north': t*0.8, 'east': t*0.1, 'yaw': 8.0 + i*0.015}
        # fake pose. Use the position monitor task
        # to get real pose in your implementation
        depth_img = np.zeros((480, 640), dtype=np.float32)
        # fake depth image replace with your depth image grabbing code
        depth_img[100:400, 200:500] = 3.0 + np.random.rand()*0.05
        # fake depth image replace with your depth image grabbing code
        mapper.update_frame(depth_img, pose)

    obstacle_points = mapper.get_global_points()

    # 2. Plan path
    planner = RRTStarPlanner(
        safety_margin=0.6,  # Drone radius + buffer
        step_size=0.8,      # Smaller = smoother but slower
        max_iter=2000
    )

    start_pt = [0.0, 0.0]
    goal_pt  = [15.0, 2.0]
    path = planner.plan(start_pt, goal_pt, obstacle_points)

    # 3. Visualize
    fig, ax = plt.subplots(figsize=(7, 10))
    if len(obstacle_points) > 0:
        ax.scatter(obstacle_points[:, 1], obstacle_points[:, 0],
                   s=3, c='gray', alpha=0.5, label='Obstacles')
    if path is not None:
        ax.plot(path[:, 1], path[:, 0], 'r-o', linewidth=2, markersize=4, label='Planned Path')
        ax.plot(start_pt[1], start_pt[0], 'go', markersize=12, label='Start')
        ax.plot(goal_pt[1], goal_pt[0], 'bx', markersize=14, label='Goal')

        # Show safety margin at waypoints
        for pt in path:
            circle = plt.Circle((pt[1], pt[0]), planner.safety_margin,
                                color='r', fill=False, linestyle='--', alpha=0.3)
            ax.add_patch(circle)

    ax.set_xlabel("East [m]"); ax.set_ylabel("North [m]")
    ax.set_title("RRT* Path on Global Point Cloud")
    ax.set_aspect('equal'); ax.grid(alpha=0.3); ax.legend()
    plt.tight_layout()
    plt.show()
```

THANK YOU